

MLOPS LAB — CS23B39 — ANSWERS

Rajalakshmi Engineering College — Dept. of CSE

IV Year CSE, VII Sem

Table of Contents

Experiment 1

Aim

To understand Git's staging and commit workflow, and to set up a local CI/CD toolchain by installing Git, Docker, and Jenkins, connecting them so a Jenkins job pulls code from Git and runs it inside a Docker container.

Algorithm / Procedure

Part a — Git staging vs committing

1. Open terminal in project folder.
2. Run `git status` to view current workspace state.
3. Run `git add index.html` to stage the specific file.
4. Run `git commit -m "message"` to permanently record the staged snapshot.
5. Run `git diff` before staging to view unstaged line-by-line changes.

Part b — Git + Docker + Jenkins integration

1. Install Git: `sudo apt install git`.
2. Install Docker: `sudo apt install docker.io`; start service.
3. Install Jenkins: add repo key, `sudo apt install jenkins`; start on port 8080.
4. Unlock Jenkins, install suggested plugins (incl. Docker Pipeline).
5. Create a new Pipeline job.
6. Configure it to pull source from a Git repository.
7. Write a Jenkinsfile that runs a build step inside a Docker container.
8. Run the job; confirm checkout + execution inside container.

Program / Code

Part a

```
git status
git add index.html
```

```
git commit -m "Add initial homepage structure to index.html"
git diff
```

Part b — Jenkinsfile

```
pipeline {
  agent {
    docker { image 'node:20-alpine' }
  }
  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url:
'https://github.com/yourusername/your-repo.git'
      }
    }
    stage('Build') {
      steps {
        sh 'node --version'
        sh 'echo "Build running inside Docker container via
Jenkins"'
      }
    }
  }
}
```

Result

Git staging/commit workflow was verified, and a working Git → Jenkins → Docker pipeline was successfully configured and executed.

Experiment 2

Aim

To establish a local, automated continuous integration process using Git and Jenkins for a static portfolio site, and to set up a Jira Scrum project with DORA metrics visualization.

Algorithm / Procedure

Part a

1. `git init`; create `index.html`; commit it.
2. Install and start Jenkins locally.
3. Create Freestyle project `Local-Portfolio-Deploy`.
4. Source Code Management → Git → local repo path.
5. Add Execute Shell build step to copy `index.html` to web root.
6. Edit `index.html`, commit the change.

7. Trigger build manually; verify deployment.

Part b

1. Create Jira Scrum project; define an Epic.
2. Break into User Stories; create and start a Sprint.
3. Map Scrum ceremonies to DevOps practices.
4. Collect sample data for 4 DORA metrics.
5. Script to calculate and visualize them.

Program / Code

Part a

```
git init
echo "<h1>My Portfolio</h1>" > index.html
git add index.html
git commit -m "Initial portfolio site"
```

Jenkins Execute Shell:

```
cp $WORKSPACE/index.html /var/www/html/index.html
```

Part b

```
import matplotlib.pyplot as plt

deployment_freq = [3, 5, 2, 6, 4, 7, 3, 5, 4, 6]
lead_time_hrs = [12, 8, 15, 6, 10, 9, 14, 7, 11, 8]
change_failure_rate = 12
mttr_hrs = [2, 1.5, 3, 1, 2.5, 1.8, 2.2, 1.1, 2.8, 1.6]

fig, axs = plt.subplots(2, 2, figsize=(10, 8))
axs[0,0].bar(range(1,11), deployment_freq, color='steelblue');
axs[0,0].set_title('Deployment Frequency')
axs[0,1].plot(range(1,11), lead_time_hrs, marker='o', color='orange');
axs[0,1].set_title('Lead Time (hrs)')
axs[1,0].bar(['CFR'], [change_failure_rate], color='crimson');
axs[1,0].set_title('Change Failure Rate (%)')
axs[1,1].plot(range(1,11), mttr_hrs, marker='s', color='green');
axs[1,1].set_title('MTTR (hrs)')
plt.tight_layout()
plt.savefig('dora_dashboard.png')
```

Result

A local Jenkins Freestyle CI job automatically deployed portfolio updates from Git, and a Jira Scrum project with DORA metrics visualization was created.

Experiment 3

Aim

To construct an automated deployment verification step using a Jenkins Declarative Pipeline that deploys a static site and validates its HTTP status, and to build a Python dashboard visualizing DORA metrics.

Algorithm / Procedure

Part a

1. Set up Git repo with web files.
2. Create Pipeline job Automated-HTTP-Validation.
3. Jenkinsfile with Deploy and Validate stages (curl for 200 OK).
4. Run pipeline; confirm success.
5. Force failure (stop server / bad port); observe Validate stage fail.
6. Restart server; re-run; confirm success.

Part b

1. Define 10+ sample points per DORA metric.
2. Compute averages.
3. Generate dashboard.

Program / Code

Part a — Jenkinsfile

```
pipeline {
  agent any
  stages {
    stage('Deploy') {
      steps {
        sh 'cp $WORKSPACE/index.html /var/www/html/index.html'
      }
    }
    stage('Validate') {
      steps {
        script {
          def status = sh(
            script: "curl -s -o /dev/null -w '%{http_code}' http://localhost",
            returnStdout: true
          ).trim()
          echo "HTTP Status: ${status}"
          if (status != '200') {
            error("Validation failed: expected 200, got ${status}")
          }
        }
      }
    }
  }
}
```

```
}
}
}
}
}
```

Part b

```
import matplotlib.pyplot as plt
import statistics

deployment_freq = [4, 6, 3, 5, 7, 2, 6, 5, 4, 8]
lead_time_hrs   = [10, 7, 14, 9, 6, 12, 8, 11, 9, 7]
cfr_percent     = [10, 15, 8, 12, 20, 5, 18, 9, 11, 14]
mttr_hrs        = [2, 1.8, 3.2, 1.5, 2.7, 1.2, 2.9, 1.6, 2.1, 1.9]

print("Avg Deployment Frequency:", statistics.mean(deployment_freq))
print("Avg Lead Time (hrs):", statistics.mean(lead_time_hrs))
print("Avg CFR (%):", statistics.mean(cfr_percent))
print("Avg MTTR (hrs):", statistics.mean(mttr_hrs))

fig, axs = plt.subplots(2, 2, figsize=(10, 8))
axs[0,0].bar(range(1,11), deployment_freq, color='steelblue');
axs[0,0].set_title('Deployment Frequency')
axs[0,1].plot(range(1,11), lead_time_hrs, marker='o', color='orange');
axs[0,1].set_title('Lead Time (hrs)')
axs[1,0].bar(range(1,11), cfr_percent, color='crimson');
axs[1,0].set_title('Change Failure Rate (%)')
axs[1,1].plot(range(1,11), mttr_hrs, marker='s', color='green');
axs[1,1].set_title('MTTR (hrs)')
plt.tight_layout()
plt.savefig('dora_dashboard_exp3.png')
```

Result

An automated Jenkins pipeline correctly deployed and validated a web page via HTTP status checks (detecting both success and forced failure), and a DORA metrics dashboard was generated.

Experiment 4

Aim

To perform collaborative Git development on a College Management System using feature branches and pull requests, and to implement GitFlow branching with a deliberate merge conflict, release tagging, and hotfix propagation.

Algorithm / Procedure

Part a

1. Pull latest project from GitHub.
2. Create branch feature/student from develop.
3. Add student.py with add/display/search functions.
4. Check, stage, commit changes.
5. Push branch; open PR into develop.

Part b

1. Set up main, develop.
2. feature/user-auth from develop; merge back.
3. release/1.0 from develop; conflicting edit on main.
4. Merge release into main; resolve conflict manually.
5. Tag v1.0 on main.
6. hotfix/login-bug from main; merge into both main and develop.
7. Push all branches and tags.

Program / Code

Part a

```
git checkout develop
git pull origin develop
git checkout -b feature/student

# student.py
students = {}

def add_student(roll_no, name, marks):
    students[roll_no] = {"name": name, "marks": marks}
    print(f"Student {name} added.")

def display_student(roll_no):
    print(students.get(roll_no, "Student not found.))

def search_student(name):
    for roll_no, info in students.items():
        if info["name"].lower() == name.lower():
            print(f"Found: {roll_no} -> {info}")
            return
    print("Student not found.")

git status
git add student.py
git commit -m "Add student.py with add, display, and search functions"
```

```
git push origin feature/student
# Open PR: feature/student -> develop on GitHub
```

Part b

```
git checkout -b develop main
git checkout -b feature/user-auth develop
echo "def login(): pass" > auth.py
git add auth.py
git commit -m "Add login skeleton"
git checkout develop
git merge feature/user-auth

git checkout -b release/1.0 develop
echo "VERSION = '1.0.0'" >> auth.py
git commit -am "Bump version for release 1.0"

git checkout main
echo "VERSION = '0.9.9'" >> auth.py
git commit -am "Hotfix version marker on main"

git checkout main
git merge release/1.0
# CONFLICT (content): Merge conflict in auth.py
# resolve manually, then:
git add auth.py
git commit -m "Resolve merge conflict between main and release/1.0"
git tag v1.0
git checkout develop
git merge release/1.0

git checkout -b hotfix/login-bug main
echo "# fixed null check" >> auth.py
git commit -am "Fix login null-check bug"
git checkout main
git merge hotfix/login-bug
git checkout develop
git merge hotfix/login-bug

git push origin main develop feature/user-auth release/1.0
hotfix/login-bug --tags
```

Result

Feature-branch collaboration with a pull request was demonstrated, and a full GitFlow workflow including a deliberately resolved merge conflict, release tagging, and hotfix propagation was successfully completed.

Experiment 5

Aim

To optimize a containerized application's footprint using Docker Multi-Stage builds, and to write a declarative Jenkinsfile with a 5-stage CI pipeline triggered via GitHub webhook.

Algorithm / Procedure

Part a

1. Create a basic Express Node.js app.
2. Write baseline single-stage Dockerfile; build as `app-large:latest`.
3. Write multi-stage Dockerfile (builder + minimal runtime); build as `app-optimized:latest`.
4. Compare sizes with `docker images`.

Part b

1. Set up Git repo with Python app and tests.
2. Jenkinsfile: Checkout → Install Deps → Unit Tests (pytest) → Static Analysis (flake8) → Deploy to Staging.
3. Configure GitHub webhook to Jenkins.
4. Push to main; confirm auto-trigger.

Program / Code

Part a

```
// server.js
const express = require('express');
const app = express();
app.get('/', (req, res) => res.send('Hello from optimized container!'));
app.listen(3000, () => console.log('Running on port 3000'));

# Dockerfile.large
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["node", "server.js"]

# Dockerfile.optimized
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY server.js .
```

```

FROM alpine:latest
RUN apk add --no-cache nodejs
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/server.js .
EXPOSE 3000
CMD ["node", "server.js"]

docker build -f Dockerfile.large -t app-large:latest .
docker build -f Dockerfile.optimized -t app-optimized:latest .
docker images | grep app-

```

Part b — Jenkinsfile

```

pipeline {
    agent any
    triggers { githubPush() }
    stages {
        stage('Checkout') {
            steps { git branch: 'main', url:
'https://github.com/you/repo.git' }
        }
        stage('Install Deps') {
            steps { sh 'pip install -r requirements.txt' }
        }
        stage('Run Unit Tests') {
            steps { sh 'pytest' }
        }
        stage('Static Analysis') {
            steps { sh 'flake8 .' }
        }
        stage('Deploy to Staging') {
            steps { sh 'echo Deploying to staging environment...' }
        }
    }
}

```

Result

Multi-stage build significantly reduced image size versus the single-stage baseline, and the 5-stage Jenkins pipeline auto-triggered successfully via GitHub webhook.

Experiment 6

Aim

To build a Jenkins pipeline that automatically builds a verified Docker container on every commit, and to create a GitHub Actions workflow enforcing test coverage on pull requests.

Algorithm / Procedure

Part a

1. Set up a static web app in a local Git repo with a Dockerfile.
2. Write a declarative Jenkinsfile with 3 stages: Checkout, Build Image, Deploy & Verify.
3. Build Image stage tags the image jenkins-app:build-\${BUILD_NUMBER}.
4. Deploy & Verify stage stops/removes any old container of the same name, runs the new one on port 8081:80, checks health.
5. Run the pipeline; verify all stages pass.

Part b

1. Create GitHub Actions workflow triggered on pull_request.
2. Run pytest with pytest-cov, enforce $\geq 80\%$ coverage.
3. Post coverage report as a PR comment; fail if under threshold.

Program / Code

Part a — Jenkinsfile

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git branch: 'main', url:
'https://github.com/you/repo.git' }
    }
    stage('Build Image') {
      steps {
        sh 'docker build -t jenkins-app:build-
${BUILD_NUMBER} .'
      }
    }
    stage('Deploy & Verify') {
      steps {
        sh '''
        docker stop jenkins-app || true
        docker rm jenkins-app || true
        docker run -d --name jenkins-app -p 8081:80 jenkins-
app:build-${BUILD_NUMBER}
        sleep 3
        curl -f http://localhost:8081 || exit 1
        '''
      }
    }
  }
}
```

Part b — .github/workflows/coverage.yml

```

name: PR Coverage Check

on:
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install -r requirements.txt pytest pytest-cov
      - name: Run tests with coverage
        run: pytest --cov=. --cov-report=xml --cov-fail-under=80
      - name: Post coverage comment
        uses: py-cov-action/python-coverage-comment-action@v3
        with:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

```

Result

A 3-stage Jenkins pipeline successfully built, deployed, and health-checked a versioned Docker image on every commit, and a GitHub Actions workflow enforced an 80% coverage gate on pull requests with automated PR comments.

Experiment 7

Aim

To securely automate Docker image publishing to a registry using Jenkins credentials, and to track multiple ML model experiments (Logistic Regression, Random Forest, XGBoost) with MLflow, selecting the best model programmatically.

Algorithm / Procedure

Part a

1. Set up a Docker Hub account or local registry (port 5000).
2. In Jenkins, add Global Credentials of type “Username with password”.
3. Write a Jenkinsfile using `withCredentials` to log in securely.
4. Build, tag (`username/repo:latest`), and push the image.

Part b

1. Load the Wine Quality dataset.

2. Train Logistic Regression, Random Forest, and XGBoost models.
3. Log hyperparameters, metrics (Accuracy, F1, ROC-AUC), and confusion matrix plots to MLflow for each run.
4. Use the MLflow Client API to query runs and select the best model by metric.

Program / Code

Part a — Jenkinsfile

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps { sh 'docker build -t username/repo:latest .' }
        }
        stage('Push to Registry') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS')]) {
                    sh 'echo $DOCKER_PASS | docker login -u
$DOCKER_USER --password-stdin'
                    sh 'docker push username/repo:latest'
                }
            }
        }
    }
}
```

Part b

```
import mlflow, mlflow.sklearn
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score,
confusion_matrix
import matplotlib.pyplot as plt

mlflow.set_experiment("Wine Quality Classification")

models = {
    "LogisticRegression": LogisticRegression(max_iter=1000),
    "RandomForest": RandomForestClassifier(n_estimators=100,
max_depth=6),
    "XGBoost": XGBClassifier(use_label_encoder=False,
eval_metric='mlogloss')
}
```

```

for name, model in models.items():
    with mlflow.start_run(run_name=name):
        model.fit(X_train, y_train)
        preds = model.predict(X_test)

        mlflow.log_param("model_type", name)
        mlflow.log_metric("accuracy", accuracy_score(y_test, preds))
        mlflow.log_metric("f1_score", f1_score(y_test, preds,
average="weighted"))
        mlflow.log_metric("roc_auc", roc_auc_score(y_test, preds,
multi_class="ovr"))

        cm = confusion_matrix(y_test, preds)
        plt.figure(); plt.imshow(cm); plt.title(f"{name} Confusion
Matrix")
        plt.savefig(f"{name}_cm.png")
        mlflow.log_artifact(f"{name}_cm.png")
        mlflow.sklearn.log_model(model, "model")

from mlflow.tracking import MlflowClient
client = MlflowClient()
runs = client.search_runs(experiment_ids=["1"],
order_by=["metrics.f1_score DESC"])
best_run = runs[0]
print("Best model run:", best_run.info.run_id,
best_run.data.metrics["f1_score"])

```

Result

A Jenkins pipeline securely authenticated and pushed a Docker image using stored credentials, and three ML models were trained and logged to MLflow with the best model selected programmatically via the Client API.

Experiment 8

Aim

To perform a complete Git collaborative workflow for an Online Bookstore application across three developers, and to define and run a 3-stage DVC pipeline versioning two dataset iterations.

Algorithm / Procedure

Part a

1. Clone the online - bookstore GitHub repository.
2. Create a feature branch (e.g., feature/cart).
3. Modify code; stage; commit.

4. Push branch; pull latest develop; resolve any conflicts.
5. Test locally; push again.
6. Open Pull Request; conduct code review; merge.

Part b

1. Define `dvc.yaml` with stages: `prepare` → `train` → `evaluate`.
2. Track dataset `v1` with `dvc add`; commit; push.
3. Run `dvc repro`.
4. Replace with dataset `v2`; `dvc add`; commit.
5. Run `dvc repro` again.
6. Compare metrics with `dvc metrics diff`.

Program / Code

Part a

```
git clone https://github.com/team/online-bookstore.git
cd online-bookstore
git checkout develop
git pull origin develop
git checkout -b feature/cart

echo "def add_to_cart(item): pass" > cart.py
git add cart.py
git commit -m "Add cart module skeleton"
git push origin feature/cart

git pull origin develop
# resolve conflicts if any, then:
git add .
git commit -m "Merge develop into feature/cart"
pytest # test
git push origin feature/cart
# Open PR: feature/cart -> develop, request review, merge after approval
```

Part b

```
# dvc.yaml
stages:
  prepare:
    cmd: python src/prepare.py
    deps: [src/prepare.py, data/raw/books.csv]
    outs: [data/processed/books_clean.csv]
  train:
    cmd: python src/train.py
    deps: [src/train.py, data/processed/books_clean.csv]
    outs: [model.pkl]
  evaluate:
```

```
cmd: python src/evaluate.py
deps: [src/evaluate.py, model.pkl]
metrics:
  - metrics.json:
      cache: false
```

```
dvc init
dvc remote add -d myremote s3://your-bucket/path
dvc add data/raw/books.csv
git add data/raw/books.csv.dvc dvc.yaml .gitignore
git commit -m "Track dataset v1 and define DVC pipeline"
dvc repro
dvc push
```

```
# --- new dataset iteration ---
cp new_books.csv data/raw/books.csv
dvc add data/raw/books.csv
git add data/raw/books.csv.dvc
git commit -m "Update dataset to v2"
dvc repro
dvc push
```

```
dvc metrics diff
```

Result

The full Git collaborative workflow (clone → branch → commit → push → PR → review → merge) was completed for the bookstore app, and a 3-stage DVC pipeline correctly reproduced results and reported metric changes between two dataset versions.

Experiment 9

Aim

To configure an Nginx reverse proxy for zero-downtime Blue-Green deployment switching, and to register, validate, and promote the best MLflow model to Production for batch inference.

Algorithm / Procedure

Part a

1. Launch Blue instance on port 8081 serving “Version 1.0 (Blue)”.
2. Launch Green instance on port 8082 serving “Version 2.0 (Green)”.
3. Configure Nginx upstream `web_service` pointing to Blue (8081).
4. Route port 80 traffic through the upstream.
5. Update upstream to Green (8082); reload Nginx (`nginx -s reload`).
6. Verify instantaneous switch with `curl`.

Part b

1. Identify best run from Experiment 7 via MlflowClient.
2. Register the model; add description and tags.
3. Transition through stages: None → Staging → Production, gated by an accuracy threshold check.
4. Load the Production model and run batch inference.

Program / Code

Part a

```
mkdir blue && echo "<h1>Version 1.0 (Blue)</h1>" > blue/index.html
mkdir green && echo "<h1>Version 2.0 (Green)</h1>" > green/index.html
(cd blue && python3 -m http.server 8081 &)
(cd green && python3 -m http.server 8082 &)
```

```
# nginx.conf
upstream web_service {
    server 127.0.0.1:8081;    # Blue
}
server {
    listen 80;
    location / {
        proxy_pass http://web_service;
    }
}
```

```
sudo nginx -t
sudo nginx -s reload
curl http://localhost      # -> Version 1.0 (Blue)

# Switch to Green: edit upstream to 127.0.0.1:8082
sudo nginx -s reload
curl http://localhost      # -> Version 2.0 (Green)
```

Part b

```
from mlflow.tracking import MlflowClient
import mlflow

client = MlflowClient()
runs = client.search_runs(experiment_ids=["1"],
order_by=["metrics.f1_score DESC"])
best_run = runs[0]

result = mlflow.register_model(f"runs://{best_run.info.run_id}/model",
"WineQualityClassifier")

client.update_model_version(
```

```

        name="WineQualityClassifier",
        version=result.version,
        description="Best model selected by F1 score"
    )
    client.set_model_version_tag("WineQualityClassifier", result.version,
                                "validated_by", "team")

    if best_run.data.metrics["accuracy"] > 0.85:

        client.transition_model_version_stage(name="WineQualityClassifier",
                                              version=result.version, stage="Staging")

        client.transition_model_version_stage(name="WineQualityClassifier",
                                              version=result.version, stage="Production")
    else:
        print("Model did not clear validation gate; staying at None
stage")

    model =
mlflow.pyfunc.load_model("models:/WineQualityClassifier/Production")
    predictions = model.predict(X_new_batch)
    print(predictions)

```

Result

Nginx successfully switched live traffic between Blue and Green instances with zero downtime, and the best MLflow model was registered, validated against an accuracy gate, promoted to Production, and used for batch inference.

Experiment 10

Aim

To perform a two-developer collaborative Git workflow on a custom project, and to build an optimized multi-stage Docker image for a Python ML service with a health check, pushed to Docker Hub and tested via curl.

Algorithm / Procedure

Part a

1. Developer A initializes repo, pushes to GitHub.
2. Developer B clones the repo.
3. Both developers create separate feature branches.
4. Each makes changes, commits, and pushes their branch.
5. Merge both branches into main via Pull Requests, resolving conflicts if needed.
6. Display final merged result with `git log --oneline --graph --all`.

Part b

1. Write a multi-stage Dockerfile using `python:3.11-slim`.
2. Install only production dependencies; copy model artifacts.
3. Add a HEALTHCHECK; expose port 8000.
4. Build, run, and verify the health check.
5. Push image to Docker Hub.
6. Test predictions via `curl`.

Program / Code

Part a

```
# Developer A
git init my-project && cd my-project
echo "# My Project" > README.md
git add README.md
git commit -m "Initial commit"
git remote add origin https://github.com/teamA/my-project.git
git push -u origin main

# Developer B
git clone https://github.com/teamA/my-project.git
cd my-project
git checkout -b feature/devB
echo "def util(): pass" > utils.py
git add utils.py
git commit -m "Add utility module"
git push origin feature/devB
# Open PR feature/devB -> main, merge after review

git log --oneline --graph --all
```

Part b

```
# ---- Build stage ----
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt
COPY model.pkl .
COPY app.py .

# ---- Runtime stage ----
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY --from=builder /app /app
ENV PATH=/root/.local/bin:$PATH
EXPOSE 8000
```

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

docker build -t username/ml-service:latest .
docker run -d -p 8000:8000 --name ml-service username/ml-
service:latest
docker ps                                # check STATUS column shows
"healthy"
docker login
docker push username/ml-service:latest
curl -X POST http://localhost:8000/predict -H "Content-Type:
application/json" -d '{"feature1": 1.2}'
```

Result

A two-developer Git workflow with branching and merging via pull requests was demonstrated, and a health-checked, multi-stage Python Docker image was built, verified, pushed to Docker Hub, and successfully tested via curl.

Experiment 11

Aim

To identify the correct Git command for reviewing uncommitted changes, and to build a FastAPI ML-serving application with Pydantic validation, tested via async integration tests inside Docker.

Algorithm / Procedure

Part a

1. Arun has cloned the project and modified `student.py`, adding a `calculate_average` function.
2. Before committing, he needs to see what changed in his working directory.
3. Use `git diff` (unstaged changes) or `git diff --staged` (after `git add`) to view line-by-line modifications.
4. Run `git status` first to confirm which files are modified.

Part b

1. Define a Pydantic `PredictionInput` / `PredictionOutput` schema.
2. Build FastAPI app with `/predict`, `/health`, `/model-info` endpoints.
3. Write async integration tests using `httpx.AsyncClient`.
4. Containerize the app; run tests inside Docker.

Program / Code

Part a

```
git status
git diff student.py

# added to student.py
def calculate_average(marks):
    return sum(marks) / len(marks)

git diff          # shows the new function as unstaged addition
git add student.py
git diff --staged  # shows the same change now staged
```

Part b

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel
import mlflow.pyfunc

app = FastAPI()
MODEL_VERSION = "1.0"
model =
mlflow.pyfunc.load_model("models:/WineQualityClassifier/Production")

class PredictionInput(BaseModel):
    feature1: float
    feature2: float
    feature3: float

class PredictionOutput(BaseModel):
    prediction: str
    confidence: float

@app.post("/predict", response_model=PredictionOutput)
def predict(data: PredictionInput):
    features = [[data.feature1, data.feature2, data.feature3]]
    pred = model.predict(features)
    return {"prediction": str(pred[0]), "confidence": 0.92}

@app.get("/health")
def health():
    return {"status": "ok", "model_version": MODEL_VERSION}

@app.get("/model-info")
def model_info():
    return {"model_name": "WineQualityClassifier", "stage":
"Production", "version": MODEL_VERSION}
```

```

# test_main.py
import pytest
from httpx import AsyncClient, ASGITransport
from main import app

@pytest.mark.asyncio
async def test_health():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport,
base_url="http://test") as client:
        response = await client.get("/health")
        assert response.status_code == 200

@pytest.mark.asyncio
async def test_predict():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport,
base_url="http://test") as client:
        response = await client.post("/predict", json={"feature1":
1.0, "feature2": 2.0, "feature3": 3.0})
        assert response.status_code == 200
        assert "prediction" in response.json()

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["pytest", "test_main.py"]

docker build -t fastapi-tests .
docker run fastapi-tests

```

Result

git diff was correctly identified and used to review Arun's uncommitted changes, and a FastAPI ML-serving app with three endpoints was built, tested asynchronously with httpx, and verified to pass all tests inside a Docker container.

Experiment 12

Aim

To perform a feature-branch GitHub workflow for a Student Attendance Management System, and to build a 4-job sequential GitHub Actions pipeline with dependency gates.

Algorithm / Procedure

Part a

1. Clone the student-attendance-system repository.
2. Check the current branch (`git branch`).
3. Create and switch to feature/student-search.
4. Create/modify Python files implementing the Student Search feature.
5. Check modified files; stage; commit with an appropriate message.
6. Push the new branch to GitHub.

Part b

1. Define a 4-job workflow: data-validation (Great Expectations), train-and-evaluate (F1 gate ≥ 0.85), build-docker (tag with Git SHA), deploy-staging (kubectl apply).
2. Chain jobs sequentially using `needs`: so each depends on the previous succeeding.

Program / Code

Part a

```
git clone https://github.com/team/student-attendance-system.git
cd student-attendance-system
git branch
git checkout -b feature/student-search

# search.py
def search_student(students, query):
    return [s for s in students if query.lower() in s["name"].lower()]

git status
git add search.py
git commit -m "Implement student search feature"
git push origin feature/student-search
```

Part b — .github/workflows/pipeline.yml

```
name: ML CI/CD Pipeline

on: [push]

jobs:
  data-validation:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install great_expectations
      - run: great_expectations checkpoint run my_checkpoint

  train-and-evaluate:
    needs: data-validation
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```

- run: python train.py
- name: Check F1 gate
  run: |
    python -c "import json,sys;
m=json.load(open('metrics.json')); sys.exit(0 if m['f1']>=0.85 else
1)"

```

build-docker:

needs: train-and-evaluate

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v4

- **run:** docker build -t myapp:\${{ github.sha }} .

deploy-staging:

needs: build-docker

runs-on: ubuntu-latest

steps:

- **run:** kubectl apply -f k8s/staging/

Result

The Student Search feature branch was correctly created, committed, and pushed following the team's Git workflow, and a 4-job GitHub Actions pipeline with sequential dependency gates (data validation → train/evaluate → build → deploy) was successfully configured.

Experiment 13

Aim

To demonstrate an automated CI/CD workflow for deploying a Student Result Web Application using Jenkins, and to provision AWS infrastructure (EC2, S3, security group) using Terraform with remote state locking.

Algorithm / Procedure

Part a

1. Set up the Student Result web app (HTML/CSS/JS) in a Git repo.
2. Configure Jenkins to pull from Git.
3. Add build steps to deploy files to a local HTTP server.
4. Trigger the pipeline and show the deployed result.

Part b

1. Write `variables.tf` with input variables (region, instance type).
2. Write `main.tf`: S3 backend with DynamoDB locking, security group (ports 22, 8000 only), EC2 t2.micro instance, S3 bucket for models.

3. Write `outputs.tf` to expose instance IP and bucket name.
4. Run `terraform init, plan, apply`.
5. Document the printed outputs.

Program / Code

Part a

```
git init student-result-app
cd student-result-app
echo "<h1>Student Results Portal</h1>" > index.html
git add . && git commit -m "Initial result portal"
```

Jenkins Freestyle/Pipeline Execute Shell:

```
cp index.html /var/www/html/index.html
```

Part b

```
# variables.tf
variable "aws_region" { default = "us-east-1" }
variable "instance_type" { default = "t2.micro" }

# main.tf
terraform {
  backend "s3" {
    bucket      = "your-terraform-state-bucket"
    key         = "mlops/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-locks"
  }
}

provider "aws" { region = var.aws_region }

resource "aws_security_group" "app_sg" {
  name = "app-sg"
  ingress { from_port = 22    to_port = 22    protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"] }
  ingress { from_port = 8000 to_port = 8000 protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"] }
  egress { from_port = 0      to_port = 0      protocol = "-1"
  cidr_blocks = ["0.0.0.0/0"] }
}

resource "aws_instance" "ml_server" {
  ami          = "ami-0c55b159cbfaffe1f0"
  instance_type = var.instance_type
  vpc_security_group_ids = [aws_security_group.app_sg.id]
  tags = { Name = "ML0ps-EC2" }
}
```

```

resource "aws_s3_bucket" "model_bucket" {
  bucket = "your-unique-ml-models-bucket-name"
}

# outputs.tf
output "instance_public_ip" { value = aws_instance.ml_server.public_ip
}
output "s3_bucket_name"      { value =
aws_s3_bucket.model_bucket.bucket }

terraform init
terraform plan
terraform apply

```

Result

A Jenkins-driven CI/CD workflow successfully built and deployed the Student Result web application, and Terraform provisioned an EC2 instance, S3 bucket, and locked-down security group with state safely stored in S3 with DynamoDB locking.

Experiment 14

Aim

To automate configuration management on the EC2 instance from Experiment 13 using an Ansible playbook with three roles (common, mlflow, dvc), verifying each service post-deployment.

Algorithm / Procedure

1. Define an inventory file pointing to the EC2 instance's public IP.
2. Create role common: installs Python 3.11, pip, git, Docker.
3. Create role mlflow: installs MLflow, deploys it as a systemd service on port 5000.
4. Create role dvc: installs DVC, configures an S3 remote.
5. Create a master playbook site.yml that applies all three roles.
6. Run ansible-playbook against the EC2 host.
7. Verify each service with `systemctl status`.

Program / Code

```

# inventory.ini
[ml_servers]
your-ec2-public-ip ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/key.pem

# roles/common/tasks/main.yml
- name: Install system packages
  apt:

```

```

    name: [python3.11, python3-pip, git, docker.io]
    state: present
    update_cache: yes
# roles/mlflow/tasks/main.yml
- name: Install MLflow
  pip:
    name: mlflow
    state: present

- name: Create MLflow systemd service
  copy:
    dest: /etc/systemd/system/mlflow.service
    content: |
      [Unit]
      Description=MLflow Tracking Server
      After=network.target

      [Service]
      ExecStart=/usr/local/bin/mlflow server --host 0.0.0.0 --port
5000
      Restart=always

      [Install]
      WantedBy=multi-user.target

- name: Start and enable MLflow service
  systemd:
    name: mlflow
    state: started
    enabled: yes
    daemon_reload: yes
# roles/dvc/tasks/main.yml
- name: Install DVC with S3 support
  pip:
    name: "dvc[s3]"
    state: present

- name: Configure DVC S3 remote
  command: dvc remote add -d myremote s3://your-bucket/path
  args:
    chdir: /home/ubuntu/project
# site.yml
- hosts: ml_servers
  become: yes
  roles:
    - common

```

```
- mlflow
- dvc
```

```
ansible-playbook -i inventory.ini site.yml
ssh ubuntu@your-ec2-public-ip "systemctl status mlflow"
```

Result

The Ansible playbook successfully configured the EC2 instance with all required packages and services across three roles, and MLflow's systemd service was confirmed running via `systemctl status`.

Experiment 15

Aim

To provision a Kubernetes cluster using Terraform and deploy a complete ML stack (ConfigMap, Deployment, Service) using Ansible/kubectl, verifying end-to-end with a live prediction request.

Algorithm / Procedure

1. Use Terraform to provision a Kubernetes cluster (Minikube for local, or EKS for cloud).
2. Define a ConfigMap for model configuration.
3. Define a Deployment running 2 replicas of the FastAPI container from Experiment 11.
4. Define a Service exposing the app on port 8000.
5. Apply all manifests with `kubectl apply`.
6. Verify pods are running with `kubectl get pods`.
7. Send a live prediction request with `curl`.

Program / Code

```
minikube start
kubectl get nodes
```

```
# configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: model-config
data:
  MODEL_VERSION: "1.0"
  MODEL_STAGE: "Production"
```

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fastapi-deployment
spec:
```

```

replicas: 2
selector:
  matchLabels: { app: fastapi }
template:
  metadata:
    labels: { app: fastapi }
  spec:
    containers:
      - name: fastapi
        image: username/fastapi-ml:latest
        ports: [{ containerPort: 8000 }]
        envFrom:
          - configMapRef: { name: model-config }

# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: fastapi-service
spec:
  selector: { app: fastapi }
  ports: [{ port: 8000, targetPort: 8000 }]
  type: NodePort

terraform apply                                # provisions cluster
ansible-playbook -i inventory.ini site.yml    # configure nodes if
needed

kubectl apply -f configmap.yaml
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

kubectl get pods
kubectl get svc

curl http://<minikube-ip>:<nodeport>/predict -X POST -d '{"feature1":
1.2}'

```

Result

Terraform provisioned the Kubernetes cluster, and Ansible/kubectl deployed the FastAPI ML service with 2 replicas behind a Service; `kubectl get pods` confirmed both replicas running, and a live `/predict` request via `curl` returned a successful prediction response, validating the end-to-end pipeline.